CS-671, Fall 2025 - HW 1, Q2

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: Brian Mason and Cursor/GPT5 -- though I do want to emphasize that the code even if LLM gernated was based on my reasoning and understanding of the problem and LLMs were simply used to help me write the code in a more efficient manner. Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

StarterCode Setup

Importing Libraries

In [4]:
```
import numpy as np
from numpy.linalg import norm
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
```

Loading Dataset, Splitting normalizer into Train and Test sets

In [5]:
```
bcd = load_breast_cancer()
X = bcd.data
y = bcd.target
y = 1 - y



# Split the dataset with a fixed random seed (random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

# so it seems we have 455 samples in the training set and 114 in the test se
```

```
(455, 30) (114, 30) (455,) (114,)
```

we have to implement our own KNN classifier and our own cross validation algorithm - we are not allowed to use sklearn cross validation library.

It seems like the breast cancer dataset contains 30 features.

The scikit-learn breast cancer dataset is imbalanced, with approximately 63% benign and 37% malignant samples.

Q2 (a): Is accuracy or F1 more appropriate?

Accuracy can be misleading on imbalanced datasets because it is dominated by the majority class. Breast cancer data is moderately imbalanced, and false negatives (predicting benign when malignant) are particularly costly. The F1 score, being the harmonic mean of precision and recall, focuses on performance for the positive class and balances missed positives (FN) and false alarms (FP). Therefore, F1 is more appropriate here.

In [6]:
```python
# Clean, minimal, well-documented implementations
from typing import Callable, Tuple

# note used SciKit Learn MinMaxScaler to write safe normalizer that does ntc

class SafeNormalizer:
    """ See for normalizing, we can either normalize the columns or the rows
    Fits on training data; transforms with feature-wise min/max.
    Prevents leakage by computing stats only on the training set passed to f
    """
    def __init__(self):
        self.data_min_: np.ndarray | None = None
        self.data_max_: np.ndarray | None = None
        self.scale_: np.ndarray | None = None
        self.min_: np.ndarray | None = None

    def fit(self, X: np.ndarray) -> "SafeNormalizer":
        self.data_min_ = X.min(axis=0)
        self.data_max_ = X.max(axis=0)
        denom = (self.data_max_ - self.data_min_) # the denominator
        # Avoid division by zero for constant features
        denom[denom == 0.0] = 1.0
        self.scale_ = 1.0 / denom
        self.min_ = -self.data_min_ * self.scale_
        return self

    def transform(self, X: np.ndarray) -> np.ndarray:
        assert self.scale_ is not None and self.min_ is not None
        return X * self.scale_ + self.min_

    def fit_transform(self, X: np.ndarray) -> np.ndarray:
        return self.fit(X).transform(X) # we first fit and than transform

# Distance metrics (vectorized over reference set)

def euclidean_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise Euclidean distances between rows of A (n_a,d) and B (n_b,d).
    # ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b
```

```
    A2 = np.sum(A*A, axis=1, keepdims=True)
    B2 = np.sum(B*B, axis=1, keepdims=True).T # we need to transpose B2
    d2 = A2 + B2 - 2 * (A @ B.T) # this is the distance metrix
    np.maximum(d2, 0.0, out=d2) # this is to make sure that the distance is
    return np.sqrt(d2) # this is the euclidean distance matrix - of dim n_a

def manhattan_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise L1 distances."""
    # Broadcast subtraction then sum abs along features
    return np.sum(np.abs(A[:, None, :] - B[None, :, :]), axis=2) # this is t

def cosine_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise cosine distance = 1 - cosine similarity."""
    A_norm = np.linalg.norm(A, axis=1, keepdims=True)
    B_norm = np.linalg.norm(B, axis=1, keepdims=True)
    # Avoid division by zero
    A_norm[A_norm == 0.0] = 1.0
    B_norm[B_norm == 0.0] = 1.0
    sim = (A @ B.T) / (A_norm * B_norm.T)
    # Numerical safety: clamp
    sim = np.clip(sim, -1.0, 1.0)
    return 1.0 - sim

class KNNClassifier:
    """Simple k-NN classifier supporting custom pairwise distance functions.
    Expects y in {0,1}. Predicts majority vote among k nearest neighbors.
    """
    def __init__(self, k: int, distance_fn: Callable[[np.ndarray, np.ndarray
        assert k >= 1
        self.k = k
        self.distance_fn = distance_fn
        self.X_train: np.ndarray | None = None
        self.y_train: np.ndarray | None = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> "KNNClassifier":
        self.X_train = X
        self.y_train = y
        return self

    def predict_proba_positive(self, X: np.ndarray) -> np.ndarray:
        assert self.X_train is not None and self.y_train is not None
        # Compute distances from queries to all training points
        D = self.distance_fn(X, self.X_train)  # shape (n_test, n_train)
        # Indices of k closest neighbors per row
        knn_idx = np.argpartition(D, kth=self.k-1, axis=1)[:, :self.k] # arg
        # For exact ordering among top-k (optional):
        # row-wise sort of the top-k slice
        row_indices = np.arange(X.shape[0])[:, None]
        knn_sorted = knn_idx[row_indices, np.argsort(D[row_indices, knn_idx]
        # Compute fraction of positive labels among top-k
        pos_counts = np.sum(self.y_train[knn_sorted], axis=1)
        return pos_counts / self.k

    def predict(self, X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
        p = self.predict_proba_positive(X)
        return (p >= threshold).astype(int)
```

```
# Metrics

def precision_recall_f1(y_true: np.ndarray, y_pred: np.ndarray) -> Tuple[flo
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    if precision + recall == 0:
        f1 = 0.0
    else:
        f1 = 2 * precision * recall / (precision + recall) # this is the har
    return precision, recall, f1
```

Q2 (b): Evaluate k=31 with Euclidean distance (normalized)

We scale with train-only MinMax, fit k-NN, and compute F1 on the held-out test set.

Q2 (b): Implement a k-NN algorithm from scratch to classify the dataset. Use `k = 31` and the `euclidean` distance, and **make sure to normalize the data**. Report your model's F1 score on the test set.

In [7]:
```
# Normalize train and test using train-only stats, then evaluate k=31
normalizer = SafeNormalizer().fit(X_train)
X_train_scaled = normalizer.transform(X_train)
X_test_scaled = normalizer.transform(X_test)

knn31 = KNNClassifier(k=31, distance_fn=euclidean_distance_matrix).fit(X_tra
y_pred_test = knn31.predict(X_test_scaled)
pre, rec, f1 = precision_recall_f1(y_test, y_pred_test)
print(f"k=31 Euclidean | Precision={pre:.4f} Recall={rec:.4f} F1={f1:.4f}")
```

```
k=31 Euclidean | Precision=1.0000 Recall=0.8810 F1=0.9367
```

k=31 Euclidean | Precision=1.0000 Recall=0.8810 F1=0.9367

Q2 (c): 5-fold cross-validation over k and distance metrics

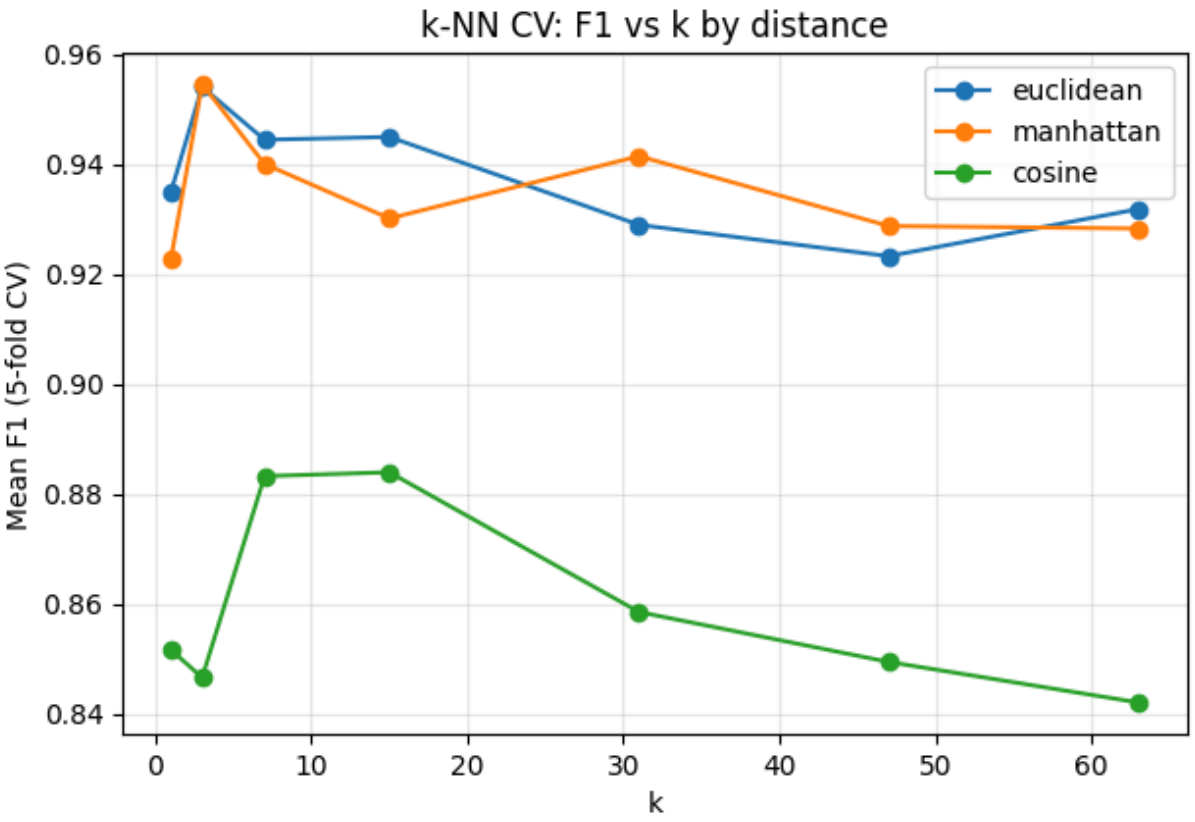
We implement CV from scratch, using only NumPy. For each distance, we sweep k and record mean F1 across folds. Then we plot F1 vs k and select the best pair.

Q2 (c): Use cross-validation with 5 folds and the F1 score to tune the value of k and the distance function used (possible distance functions to use could be Euclidean distance, Manhattan distance, or cosine similarity). Make sure to use at least five values of k between 1 and 63, and try at least two distance functions. For each distance function, show a plot where the x-axis depicts the value of k and the y-axis depicts the average F1 score for that value of k during cross-validation. Which pair of parameters performed the best? Note: You may find the array split method from NumPy helpful when implementing cross-validation.

In [16]:

```python
# 5-fold CV implementation (train set only) and plotting
from numpy import array_split # useful for splitting the data into folds

def kfold_indices(n, n_splits, rng):
    idx = np.arange(n)
    if rng is not None:
        rs = np.random.RandomState(rng)
        rs.shuffle(idx)
    folds = array_split(idx, n_splits)
    for i in range(n_splits):
        val_idx = folds[i]
        train_idx = np.concatenate([folds[j] for j in range(n_splits) if j !
        yield train_idx, val_idx

k_values = [1, 3, 7, 15, 31, 47, 63]
distance_fns = {
    "euclidean": euclidean_distance_matrix,
    "manhattan": manhattan_distance_matrix,
    "cosine": cosine_distance_matrix,
}

# Normalize once using full train stats? I don't think that's a good idea -
# To avoid leakage, each fold fits scaler on fold-train only.
def cv_mean_f1_for_distance(distance_name: str, distance_fn: Callable[[np.nd

    mean_f1_per_k: list[float] = []

    for k in k_values:
        f1_scores: list[float] = []
        for tr_idx, va_idx in kfold_indices(len(X_train), n_splits=5, rng=42
            X_tr, X_va = X_train[tr_idx], X_train[va_idx]
            y_tr, y_va = y_train[tr_idx], y_train[va_idx]
            # Fit scaler on fold-train only
            scaler_cv = SafeNormalizer().fit(X_tr)
            X_tr_s = scaler_cv.transform(X_tr)
            X_va_s = scaler_cv.transform(X_va)
            # Train and eval
            model = KNNClassifier(k=k, distance_fn=distance_fn).fit(X_tr_s,
            y_va_pred = model.predict(X_va_s)
            _, _, f1_va = precision_recall_f1(y_va, y_va_pred)
            f1_scores.append(f1_va)
        mean_f1_per_k.append(float(np.mean(f1_scores)))
    return k_values, mean_f1_per_k

results = {}
plt.figure(figsize=(7,4.5))
# used chatgpt to help me plot this better
for name, fn in distance_fns.items():
    ks, mean_f1s = cv_mean_f1_for_distance(name, fn)
    results[name] = {"k": ks, "mean_f1": mean_f1s}
    plt.plot(ks, mean_f1s, marker='o', label=name)
plt.xlabel('k')
plt.ylabel('Mean F1 (5-fold CV)')
plt.title('k-NN CV: F1 vs k by distance')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.show()

# Find best pair
best_name, best_k, best_score = None, None, -1.0
for name, res in results.items():
    for k, s in zip(res["k"], res["mean_f1"]):
        if s > best_score:
            best_name, best_k, best_score = name, k, s
print(f"Best by CV: distance={best_name}, k={best_k}, mean F1={best_score:.4
```




```
Best by CV: distance=manhattan, k=3, mean F1=0.9546
```

Q2 (d): Using the best parameters determined in part (c), report the performance of a k-NN classifier on the test set. Compare this to your model in part (b). Is it as you expected?

Awesome, so it seems that the best metrics are using K = 3 and the manhattan distance.

In [20]:
```
# Refit scaler on full training set and evaluate best model on test
# Reuse `results` and `best_name`, `best_k` computed above.
best_distance_fn = distance_fns['manhattan']
best_k = 3

scaler_best = SafeNormalizer().fit(X_train)
X_train_best = scaler_best.transform(X_train)
X_test_best = scaler_best.transform(X_test)

best_model = KNNClassifier(k=best_k, distance_fn=best_distance_fn).fit(X_tra
y_test_pred_best = best_model.predict(X_test_best)
```

```
pre_b, rec_b, f1_b = precision_recall_f1(y_test, y_test_pred_best)
print(f"Best params on test | distance={best_name}, k={best_k} -> Precision=

# Brief comparison with part (b)
print("Compared to fixed k=31 Euclidean above.")
```

```
Best params on test | distance=manhattan, k=3 -> Precision=1.0000 Recall=0.9
286 F1=0.9630
Compared to fixed k=31 Euclidean above.
```

Best parameters on the test set: distance = Manhattan, k = 3, resulting in Precision = 1.0000, Recall = 0.9286, F1 score = 0.9630.

When comparing the two models, the version with k = 31 and Euclidean distance exhibited very high precision but missed more malignant cases, which lowered its recall. After tuning with cross-validation, the choice of k = 3 and Manhattan distance performed better because a smaller k makes the classifier more sensitive to local data structures, and Manhattan distance can sometimes more effectively capture differences between samples when features vary across scales. This makes sense given that the data is imbalanced; a smaller k is more likely to be influenced by local structures. From the graph, we can see that Manhattan and Euclidean distances are quite similar, making it interesting that we achieve better performance with Manhattan distance compared to angle metrics like cosine similarity.

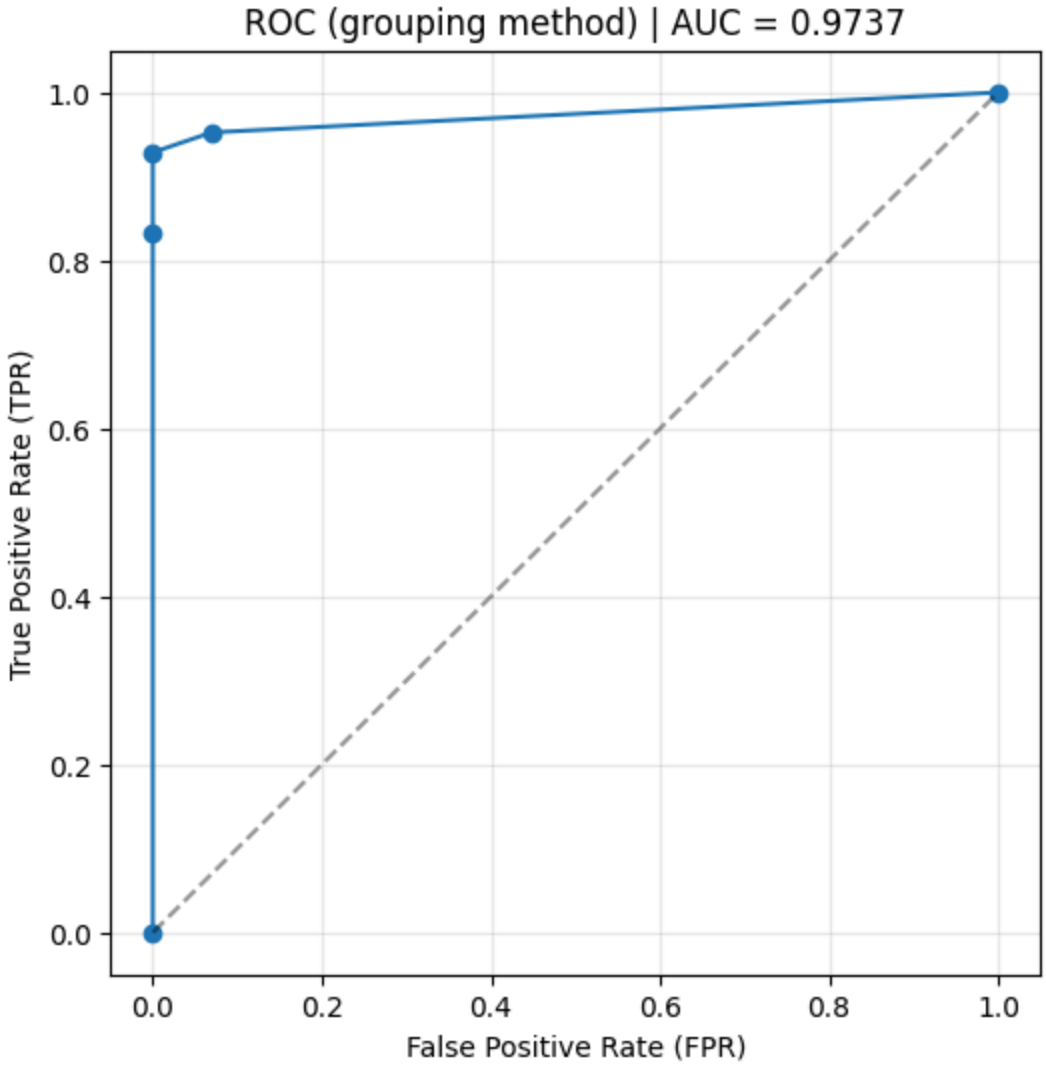
Q2 (e): ROC curve and AUC (from scratch)

We compute positive-class vote fractions, sweep thresholds, compute (FPR, TPR), plot ROC, and calculate AUC using trapezoidal rule. The grouping-by-vote approach is used for efficiency.

Q2 (e -> i): Using the test dataset and the same parameters as part (c), plot the ROC curve of your classifier as points connected by lines. You may implement the ROC curve computation using either of the two methods discussed in the HW PDF

In [25]:
```
# --- Elegant grouping ROC for k-NN (approach 2) ---

def roc_by_grouping(y_true: np.ndarray, proba_pos: np.ndarray, k: int) -> tu
    """
    Build ROC by grouping points with exactly i positive neighbors (i in {0.
    We sweep from i=k down to 0, accumulating TP and FP.
    """
    y_true = np.asarray(y_true)
    proba_pos = np.asarray(proba_pos)

    # Recover integer neighbor counts; guard tiny float noise.
    # proba_pos should be exactly {0/k,1/k,...,k/k} for majority-vote k-NN.
    kpos = np.rint(proba_pos * k).astype(int)
    kpos = np.clip(kpos, 0, k)

    P = int(np.sum(y_true == 1))
```

```
    N = int(np.sum(y_true == 0))

    # Start at (0,0): predicting nothing positive
    tprs = [0.0]
    fprs = [0.0]
    tp = 0
    fp = 0

    # Add groups from i = k down to 0 - here k is 3
    for i in range(k, -1, -1):
        mask = (kpos == i)
        if np.any(mask):
            tp += int(np.sum(y_true[mask] == 1))
            fp += int(np.sum(y_true[mask] == 0))
            tprs.append(tp / P if P else 0.0)
            fprs.append(fp / N if N else 0.0)

    # Ensure we end at (1,1) (in case of any numerical weirdness)
    tprs[-1] = 1.0 if P else tprs[-1]
    fprs[-1] = 1.0 if N else fprs[-1]

    return np.array(fprs), np.array(tprs)


def auc_trapezoid(x: np.ndarray, y: np.ndarray) -> float:
    """Simple trapezoidal AUC (assumes x is sorted ascending)."""
    area = 0.0
    for i in range(1, len(x)):
        dx = x[i] - x[i-1] # this is the marginal x
        area += dx * (y[i] + y[i-1]) / 2.0 # this is the area of the trapezo
    return float(area)


proba_pos = best_model.predict_proba_positive(X_test_best)
fpr_g, tpr_g = roc_by_grouping(y_test, proba_pos, 3) # our best k is 3

# Sort by FPR just in case equal-FPR plateaus appear out of order
order = np.argsort(fpr_g)
fpr_g, tpr_g = fpr_g[order], tpr_g[order]

auc_g = auc_trapezoid(fpr_g, tpr_g)

plt.figure(figsize=(6,6))
plt.plot(fpr_g, tpr_g, marker='o', lw=1.5)
plt.plot([0,1], [0,1], 'k--', alpha=0.4)
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'ROC (grouping method) | AUC = {auc_g:.4f}')
plt.grid(True, alpha=0.3)
plt.show()

print(f"AUC (grouping method, from scratch) = {auc_g:.6f}")
```




```
AUC (grouping method, from scratch) = 0.973710
```